

Fase 3 - Arquitetura MVC (Controller, Service, Repository)

Esse material cobre a divisão em camadas usada em aplicações backend: Controller, Service e Repository. Os exemplos aqui ainda são em Java puro, sem Spring — a ideia é entender **o porquê** da separação antes de ver como o Spring facilita isso (isso vem na Fase 4).

1. O que é MVC, e por que backend usa uma variação dele

MVC significa **Model - View - Controller**, um padrão criado originalmente pra aplicações com tela:

- **Model:** os dados e as regras sobre eles.
- **View:** a tela que o usuário vê.
- **Controller:** recebe a ação do usuário e decide o que fazer.

Em uma API backend (o que vocês fazem no dia a dia), não existe uma “tela” — quem consome a API é outro sistema, um app mobile, um front-end separado, etc. Por isso, na prática, o backend usa uma variação do MVC com três camadas mais específicas:

- **Controller:** recebe a requisição (HTTP, por exemplo) e devolve uma resposta.
- **Service:** contém a regra de negócio.
- **Repository:** acessa os dados (banco de dados, arquivo, outra API).

O “Model” continua existindo, mas normalmente é só a classe que representa os dados (a entidade), sem lógica pesada dentro dela.

2. O papel de cada camada

Model (ou Entity): representa uma “coisa” do seu domínio (um Produto, um Cliente, um Pedido). Geralmente só tem atributos e getters/setters, sem regra de negócio.

```
public class Produto {  
    private Long id;  
    private String nome;  
    private double preco;  
  
    // getters e setters  
}
```

Repository: responsável por buscar, salvar, atualizar e apagar dados. Não deveria conter regra de negócio — só sabe “conversar” com a fonte de dados.

```

public class ProdutoRepository {
    private List<Produto> dados = new ArrayList<>();

    public void salvar(Produto produto) {
        dados.add(produto);
    }

    public List<Produto> listarTodos() {
        return dados;
    }

    public Produto buscarPorId(Long id) {
        return dados.stream()
            .filter(p -> p.getId().equals(id))
            .findFirst()
            .orElse(null);
    }
}

```

Service: contém a regra de negócio. É aqui que ficam as validações, cálculos e decisões — o Service usa o Repository, mas não é o Repository.

```

public class ProdutoService {
    private ProdutoRepository repository;

    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }

    public void cadastrar(Produto produto) {
        if (produto.getPreco() < 0) {
            throw new IllegalArgumentException("Preco nao pode ser negativo");
        }
        repository.salvar(produto);
    }

    public List<Produto> listarTodos() {
        return repository.listarTodos();
    }
}

```

Controller: ponto de entrada. Recebe a requisição, chama o Service e devolve a resposta. Não deveria ter regra de negócio nem acessar o Repository diretamente.

```

public class ProdutoController {
    private ProdutoService service;

    public ProdutoController(ProdutoService service) {
        this.service = service;
    }
}

```

```

public void cadastrarProduto(Produto produto) {
    service.cadastrar(produto);
    System.out.println("Produto cadastrado com sucesso");
}

public void listarProdutos() {
    List<Produto> produtos = service.listarTodos();
    produtos.forEach(p -> System.out.println(p.getNome() + " - R$" + p.getPreco()));
}
}

```

3. Por que não colocar tudo numa classe só

Veja como ficaria **sem** separar em camadas — tudo misturado:

```

public class ProdutoTudoJunto {
    private List<Produto> dados = new ArrayList<>();

    public void cadastrarProduto(Produto produto) {
        // validacao de negocio misturada com acesso a dado
        if (produto.getPreco() < 0) {
            System.out.println("Erro: preco invalido");
            return;
        }
        dados.add(produto);
        System.out.println("Produto cadastrado com sucesso");
    }
}

```

Funciona, mas traz problemas conforme o projeto cresce:

- Se amanhã você trocar a “lista em memória” por um banco de dados de verdade, precisa mexer na mesma classe que tem a regra de negócio — risco de quebrar as duas coisas juntas.
- Fica difícil testar só a regra de negócio sem depender de banco de dados.
- Se outra parte do sistema (por exemplo, um processo agendado) precisar cadastrar produto sem passar por uma requisição HTTP, não dá pra reaproveitar — a regra está grudada no Controller.

Separando em camadas, cada uma muda por um motivo diferente: o Repository muda se você trocar a forma de guardar dado; o Service muda se a regra de negócio mudar; o Controller muda se a forma de receber a requisição mudar (por exemplo, de REST pra outro protocolo).

4. Regra prática pra não errar a camada

Quando você estiver decidindo onde colocar um trecho de código, pergunte:

- “Isso é sobre **receber/devolver** algo?” → Controller
- “Isso é uma **regra/decisão** do negócio?” → Service

- “Isso é sobre **buscar/gravar** dado?” → Repository

Um erro comum de quem está começando é colocar validação de negócio no Controller (porque “é ali que os dados chegam primeiro”). O ponto de entrada não é o lugar da decisão — ele só recebe e repassa.

Exercício 3.1 — Identificar a camada errada O código abaixo tem uma regra de negócio no lugar errado. Identifique qual é o problema e reescreva colocando o código na camada correta.

```
public class ClienteController {
    private ClienteRepository repository;

    public void cadastrarCliente(Cliente cliente) {
        if (cliente.getIdade() < 18) {
            System.out.println("Cliente deve ser maior de idade");
            return;
        }
        repository.salvar(cliente);
    }
}
```

Exercício 3.2 — Construir as três camadas do zero Modele um cadastro de Cliente com os atributos id, nome e idade, seguindo a mesma estrutura do exemplo de Produto: 1. Crie a classe Cliente (Model). 2. Crie ClienteRepository com métodos para salvar, listar todos e buscar por id. 3. Crie ClienteService com uma regra de negócio: não permitir cadastrar cliente menor de 18 anos (lance uma exceção, como no exemplo de Produto). 4. Crie ClienteController que usa o Service pra cadastrar e listar clientes, imprimindo os resultados no console. 5. No main, monte a cadeia (Controller usando Service, Service usando Repository) e teste cadastrando um cliente válido e um inválido.

Exercício 3.3 — Reaproveitamento entre camadas Usando o ClienteService do exercício anterior, escreva um segundo “ponto de entrada” que **não** é um Controller — pode ser um método main separado, simulando um processo automático que cadastra um cliente sem passar por nenhuma “requisição”. Chame o mesmo ClienteService usado pelo Controller. Depois, responda em 2-3 linhas: isso seria possível se a regra de validação estivesse dentro do Controller em vez de estar no Service?

O que vem a seguir

Na Fase 4, você vai ver como o Spring Boot facilita exatamente essa separação com anotações (@RestController, @Service, @Repository) — o papel de cada camada continua **exatamente o mesmo** que você acabou de praticar aqui, só muda a forma de declarar isso pro framework.